

Types of Function Arguments in Python

In Python, **user-defined functions** can accept different types of arguments.

These argument types follow **pre-defined rules** provided by Python, which help us design flexible and reusable functions

Python mainly supports **four types of function arguments**:

1. Required Arguments (Positional Arguments)

- Required arguments are **mandatory**.
- Values must be passed:
 - In the **correct number**
 - In the **correct order**
- If order or count is wrong, Python raises an error.

Syntax

```
def FunctionName(arg1, arg2):  
    statements
```

Example

```
def Batches1(name, fees):  
    print("Batch name is", name)  
    print("Fees are", fees)
```

```
Batches1("Python", 28000)
```

Important Points

- Position matters.
- Arguments are matched **based on position**, not name.
- Swapping values leads to logical errors.

2. Keyword Arguments

- Arguments are passed using **parameter names (keywords)**.
- Order **does not matter**.
- Improves **code readability**.

Syntax

FunctionName(parameter=value)

Example

```
def Batches2(name, fees):  
    print("Batch name is", name)  
    print("Fees are", fees)  
  
Batches2(fees=27000, name="PPA")  
Batches2(name="LB", fees=28000)
```

Advantages

- No need to remember order
- Clear mapping between parameter and value
- Less chance of mistakes

3. Default Arguments

- A default value is assigned to a parameter.
- If value is **not provided during function call**, default value is used.
- Default arguments must be placed **after required arguments**.

Syntax

```
def FunctionName(arg1, arg2=default_value):  
    statements
```

Example

```
def Batches3(name, fees=25000):  
    print("Batch name is", name)  
    print("Fees are", fees)
```

```
Batches3("React", 28000)  
Batches3("React")  
Batches3(name="LB")
```

Important Rules

- Default arguments are evaluated **once**.
- Required arguments cannot follow default arguments.

4. Variable Number of Arguments (*args)

- Used when number of arguments is **not fixed**.
- All values are stored as a **tuple**.
- Represented using *****.

Syntax

```
def FunctionName(*args):  
    statements
```

Example

```
def Add(*no):  
    ans = 0  
    for i in no:  
        ans = ans + i  
    return ans  
  
print(Add(10, 20, 30))  
print(Add(10, 20, 30, 40, 50))
```

Use Case

- Mathematical operations
- Flexible input handling